WEB PROGRAMMING (WP) · MINI PROJECT REPORT

Campus EventHub

A full-stack college event registration website

HTML5 · CSS3 · JavaScript · Node.js 24 · Express 5 · SQLite · REST API



Course	TY BTech · Web Programming
Submitted by	
Roll No. / Division	
Submitted to	
Source code	github.com/chhelu123/campus-eventhub

Contents: 1 Aim · 2 Description · 3 Features · 4 Technology Stack · 5 Architecture · 6 Database Schema · 7 API Endpoints · 8 Code · 9 How to Run · 10 Output · 11 Conclusion

1. Aim

To design and develop a full-stack, database-backed web application using **HTML, CSS and JavaScript** on the client side and **Node.js with Express** and an **SQLite** database on the server side, communicating through a **REST API** that exchanges JSON.

2. Description

Colleges run many events every semester - hackathons, cultural nights, sports leagues, workshops and guest talks. Registrations are usually collected on paper or through scattered online forms, which makes it hard for students to see what is coming up and hard for organisers to know how many seats are left.

Campus EventHub brings all of this into one website. **Students** sign up, browse and search events, filter them by category and date, open an event to read the details, register with one click when seats are available and cancel if their plans change. A *My Registrations* page lists everything they have signed up for. The **admin** (event coordinator) logs in to an admin panel to create, edit and delete events, see overall statistics and view or download the list of students registered for each event.

All rules are enforced on the server - seat capacity, one registration per student per event, no registration for past events and admin-only actions - so the data stays consistent even if someone calls the API directly.

3. Features

Area	Features
Authentication	Sign up, log in, log out; passwords hashed with bcrypt; login kept in an httpOnly JWT cookie (7 days); role-based access (student / admin)
Browse events	Event cards with category colour, date badge, venue, time and live seat bar; search by title / description / venue / organiser; category filter; date-range filter; Upcoming / Past / All toggle
Event details	Description, date, time, venue, organiser, seats registered / left; Register and Cancel buttons with instant update
My registrations	Upcoming and Past tabs, cancel a registration, link to each event
Admin panel	Statistics cards; table of all events; create / edit / delete events in a modal form; registrant list per event with CSV download
Validation	Client-side validation with inline messages; the same rules re-checked on the server with proper HTTP status codes (400, 401, 403, 404, 409)
User interface	Modern, responsive design (desktop, tablet, mobile); toast notifications; loading and empty states

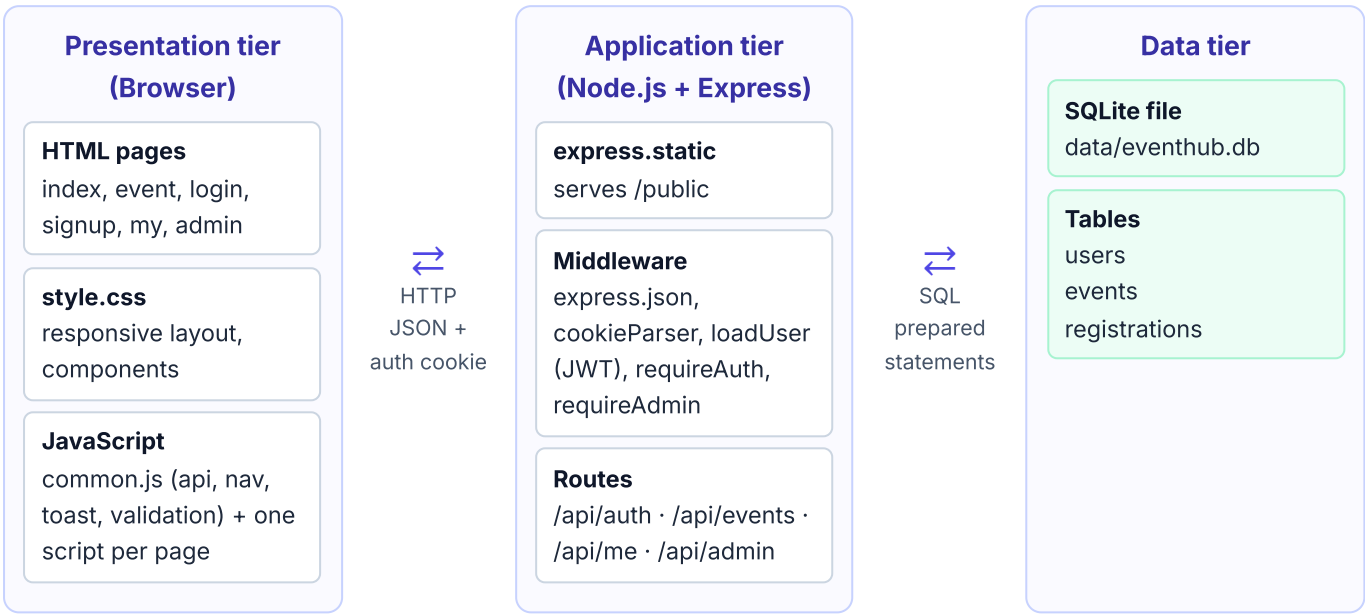
Demo accounts (created automatically on first run): admin `admin@eventhub.com` / `admin123`; sample students such as `aarav@student.edu` / `student123`.

4. Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3 (Flexbox, Grid, media queries), vanilla JavaScript, `fetch()`	Pages, styling, responsive layout, calling the API and updating the DOM
Backend	Node.js 24, Express 5	Web server, REST API routing, middleware, serving static files
Database	SQLite through Node's built-in `node:sqlite` (`DatabaseSync`)	Relational storage in a single file; no separate DB server or native add-on
Security	bcryptjs, jsonwebtoken, cookie-parser	Password hashing, signed login tokens, reading the auth cookie
Tools	Git / GitHub, Google Chrome, curl, VS Code	Version control, testing, screenshots, API testing

5. System Architecture

The application follows a three-tier **client-server architecture**. The browser loads static HTML, CSS and JavaScript from the Express server; the JavaScript then calls the REST API with `fetch()`. The server authenticates the request, applies the business rules and reads or writes the SQLite database, and sends back JSON that the page renders.



Request flow example - student registers for an event

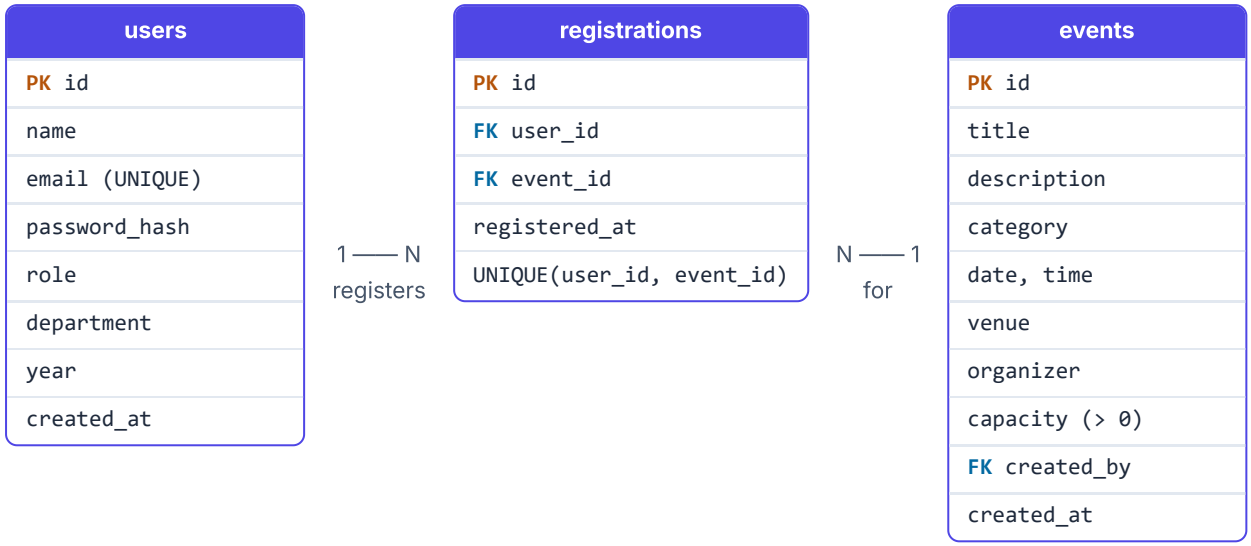
1. The student clicks *Register now*; `event.js` sends `POST /api/events/1/register` with the login cookie.
2. `loadUser` verifies the JWT and sets `req.user`; `requireAuth` lets the request through.
3. The route checks the event exists, is upcoming, is not full and the student is not already registered, then inserts a row in `registrations`.
4. The server replies `201 Created` with the updated event; the page shows a toast and redraws the seat bar.

Folder structure

```
campus-eventhub/
├── package.json            dependencies and npm scripts
├── server/
│   ├── index.js            Express app: middleware, routes, static files, errors
│   ├── db.js               SQLite connection, CREATE TABLE, seed data
│   ├── middleware/auth.js  JWT cookie helpers, requireAuth, requireAdmin
│   └── routes/             auth.js, events.js, me.js, admin.js
├── public/
│   ├── *.html              index, event, login, signup, my, admin, 404
│   ├── css/style.css
│   └── js/                 common.js, events.js, event.js, auth.js, my.js, admin.js
├── screenshots/            output screenshots
└── report/                 this report (HTML source of REPORT.pdf)
```

6. Database Schema

The database has three tables. `registrations` is the junction table of the many-to-many relationship between students and events.



SQL (from server/db.js)

```
CREATE TABLE users (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  name          TEXT NOT NULL,
  email         TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  role          TEXT NOT NULL DEFAULT 'student' CHECK (role IN ('student','admin')),
  department    TEXT,
  year          TEXT,
  created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE events (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT NOT NULL,
  description TEXT NOT NULL,
  category    TEXT NOT NULL,
  date        TEXT NOT NULL,          -- YYYY-MM-DD
  time        TEXT NOT NULL,          -- HH:MM
  venue       TEXT NOT NULL,
  organizer   TEXT NOT NULL,
  capacity    INTEGER NOT NULL CHECK (capacity > 0),
  created_by  INTEGER REFERENCES users(id) ON DELETE SET NULL,
  created_at  TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE registrations (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id       INTEGER NOT NULL REFERENCES users(id)  ON DELETE CASCADE,
  event_id      INTEGER NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  registered_at TEXT NOT NULL DEFAULT (datetime('now')),
  UNIQUE (user_id, event_id)
);
```

- **Primary keys** identify each row; **foreign keys** link registrations to users and events.
- `ON DELETE CASCADE` removes an event's registrations when the event is deleted.
- `UNIQUE (user_id, event_id)` stops a student from registering twice; `CHECK` constraints guard the role and capacity values.

7. REST API Endpoints

All endpoints are under `/api`, accept and return JSON, and report errors as `{"error": "message"}` with a suitable HTTP status code.

Method	Endpoint	Access	Description
POST	`/api/auth/signup`	Public	Create a student account and log in
POST	`/api/auth/login`	Public	Log in with email and password (sets cookie)
POST	`/api/auth/logout`	Public	Clear the login cookie
GET	`/api/auth/me`	Public	Current user, or null
GET	`/api/events`	Public	List events; query: search, category, from, to, when
GET	`/api/events/categories`	Public	Allowed categories
GET	`/api/events/:id`	Public	One event with seats left / registered flags
POST	`/api/events/:id/register`	Logged in	Register for an event
DELETE	`/api/events/:id/register`	Logged in	Cancel own registration
GET	`/api/me/registrations`	Logged in	My registered events
POST	`/api/events`	Admin	Create an event
PUT	`/api/events/:id`	Admin	Update an event
DELETE	`/api/events/:id`	Admin	Delete an event (cascades registrations)
GET	`/api/events/:id/registrants`	Admin	Students registered for an event
GET	`/api/admin/stats`	Admin	Dashboard totals

Status codes used: 200 OK, 201 Created, 400 Bad Request (validation), 401 Unauthorized (not logged in), 403 Forbidden (not admin), 404 Not Found, 409 Conflict (already registered, event full, email taken).

8. Key Code with Explanation

8.1 Server setup - server/index.js

```
const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json({ limit: '100kb' }));   // parse JSON request bodies
app.use(cookieParser());                     // parse the auth cookie
app.use(loadUser);                           // attach req.user when logged in

// REST API
app.use('/api/auth', require('./routes/auth'));
app.use('/api/events', require('./routes/events'));
app.use('/api/me', require('./routes/me'));
app.use('/api/admin', require('./routes/admin'));

// Frontend (HTML/CSS/JS)
app.use(express.static(path.join(__dirname, '..', 'public'), { extensions: ['html'] }));

app.listen(PORT, () => console.log(`Campus EventHub running at http://localhost:${PORT}`));
```

> **Explanation:** Express passes every request through a chain of **middleware**: the JSON body and cookies are parsed, then `loadUser` checks the login cookie. Paths starting with `/api` go to the route modules (the REST API); everything else is served from `public/` as static files.

8.2 Database - server/db.js

```
const { DatabaseSync } = require('node:sqlite');
const db = new DatabaseSync(DB_PATH);
db.exec('PRAGMA foreign_keys = ON;');

db.exec(`
  CREATE TABLE IF NOT EXISTS registrations (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id       INTEGER NOT NULL REFERENCES users(id)  ON DELETE CASCADE,
    event_id      INTEGER NOT NULL REFERENCES events(id) ON DELETE CASCADE,
    registered_at TEXT    NOT NULL DEFAULT (datetime('now')),
    UNIQUE (user_id, event_id)
  );
`);
```

> **Explanation:** `node:sqlite` is SQLite built into Node.js, so no database server or native add-on is needed. `CREATE TABLE IF NOT EXISTS` creates the tables on the first run. Foreign keys with `ON DELETE CASCADE` and `UNIQUE (user_id, event_id)` keep the data consistent, and all queries use **prepared statements** with placeholders, which prevents SQL injection.

8.3 Authentication - routes/auth.js and middleware/auth.js

```
// Login route
router.post('/login', (req, res) => {
  const email = String(req.body.email || '').trim().toLowerCase();
  const password = String(req.body.password || '');
  const row = db.prepare('SELECT * FROM users WHERE email = ?').get(email);
  if (!row || !bcrypt.compareSync(password, row.password_hash)) {
    return res.status(401).json({ error: 'Invalid email or password.' });
  }
  const { password_hash, created_at, ...user } = row;
  issueToken(res, user);
  res.json({ user });
});

// Middleware
function issueToken(res, user) {
  const token = jwt.sign({ id: user.id, role: user.role }, JWT_SECRET, { expiresIn: '7d' });
  res.cookie('eh_token', token, { httpOnly: true, sameSite: 'lax', maxAge: 7 * 24 * 60 * 60 * 1000 });
}

function requireAdmin(req, res, next) {
  if (!req.user) return res.status(401).json({ error: 'Please log in to continue.' });
  if (req.user.role !== 'admin') return res.status(403).json({ error: 'Admin access only.' });
  next();
}
```

> **Explanation:** At sign-up the password is hashed with **bcrypt**, so the real password is never stored. At login `bcrypt.compareSync` checks it; on success a **JSON Web Token** (user id + role, signed with a secret) is stored in an **httpOnly cookie** that page scripts cannot read. `requireAuth` / `requireAdmin` protect routes and answer 401 / 403.

8.4 A route with business rules - registering for an event

```
router.post('/:id/register', requireAuth, (req, res) => {
  const event = getEvent(req.params.id, req.user.id);
  if (!event) return res.status(404).json({ error: 'Event not found.' });
  if (event.is_past) return res.status(400).json({ error: 'This event has already taken place.' });
  if (event.is_registered) return res.status(409).json({ error: 'You are already registered for this
event.' });
  if (event.seats_left <= 0) return res.status(409).json({ error: 'Sorry, this event is full.' });

  db.prepare('INSERT INTO registrations (user_id, event_id) VALUES (?, ?)').run(req.user.id, event.id);
  res.status(201).json({ message: 'Registered successfully.', event: getEvent(event.id, req.user.id)
});
});
```

> **Explanation:** `getEvent` loads the event with a live registration count. The route checks the business rules in order - event exists, not in the past, not already registered, seats left - then inserts the row and returns `201 Created` with the updated event so the page refreshes the seat count without reloading.

8.5 Frontend fetch() - public/js/common.js and event.js

```
// Reusable wrapper around fetch()
async function api(path, { method = 'GET', body } = {}) {
  const res = await fetch('/api' + path, {
    method,
    credentials: 'same-origin', // send the auth cookie
    headers: body ? { 'Content-Type': 'application/json' } : {},
    body: body ? JSON.stringify(body) : undefined,
  });
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(data.error || `Request failed (${res.status})`);
  return data;
}

// Register button on the event details page
async function act(method, btn, okMsg) {
  btn.disabled = true;
  try {
    const { event } = await api(`/events/${eventId}/register`, { method });
    toast(okMsg);                 // "You are registered! See you there."
    render(event, currentUser);   // re-draw seats and button
  } catch (err) {
    toast(err.message, 'error');  // e.g. "Sorry, this event is full."
    btn.disabled = false;
  }
}
```

> **Explanation:** All pages use the small `api()` helper around `fetch()`: it sends JSON with the login cookie, parses the response and turns error responses into JavaScript errors carrying the server message. The Register button calls `POST` (or `DELETE` to cancel) and re-renders the page with the returned data, showing a toast message.

8.6 Client-side form validation - public/js/auth.js

```
const ok = validateForm(signup, {
  name: [[(v) => v.length >= 2, 'Please enter your full name']],
  email: [[(v) => v, 'Email is required'], [isEmail, 'Enter a valid email address']],
  department: [[(v) => v, 'Choose your department']],
  year: [[(v) => v, 'Choose your year']],
  password: [[(v) => v.length >= 6, 'Password must be at least 6 characters']],
  confirm: [[(v, f) => v === f.password.value, 'Passwords do not match']],
});
if (ok) submitWith(signup, '/auth/signup', { /* form values */ });
```

> **Explanation:** Each form field has a list of rules (a test function and a message). `validateForm` highlights invalid fields and shows the message under them, catching mistakes instantly. The server repeats the important checks because client-side validation can be bypassed.

9. How to Run

Requirement: Node.js 22.13 or newer (developed on Node.js 24). SQLite is built into Node, so nothing else needs to be installed.

```
git clone https://github.com/chhelu123/campus-eventhub.git
cd campus-eventhub
npm install
npm start          # then open http://localhost:3000
```

- On the first run `data/eventhub.db` is created with the admin, 5 sample students, 8 events and sample registrations (event dates are relative to the first run).
- Admin login: `admin@eventhub.com` / `admin123`. Delete the `data/` folder to reset the database.
- Optional environment variables: `PORT`, `JWT_SECRET`, `DB_PATH`.

10. Output

The screenshots below were captured from the running application in Google Chrome (1366 × 850) while performing each action: signing up a student, registering for events, logging in as admin and creating an event.

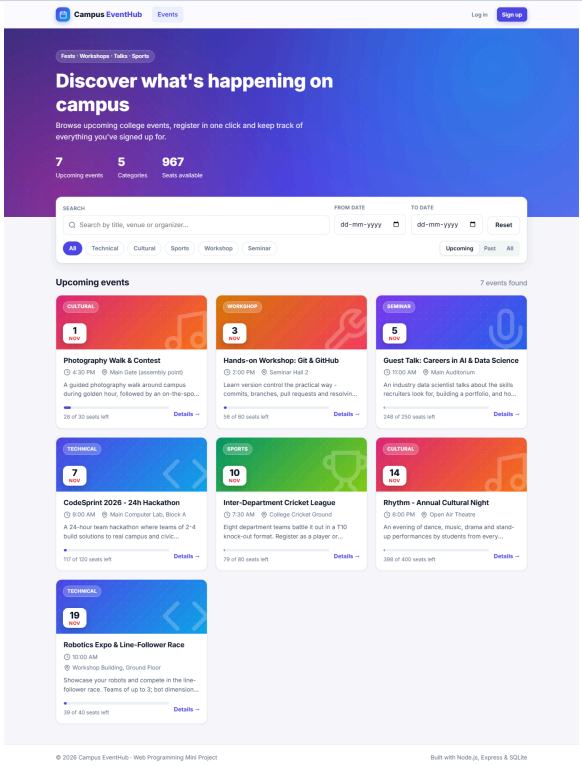


Figure 1: Home page - list of upcoming events with search and filters

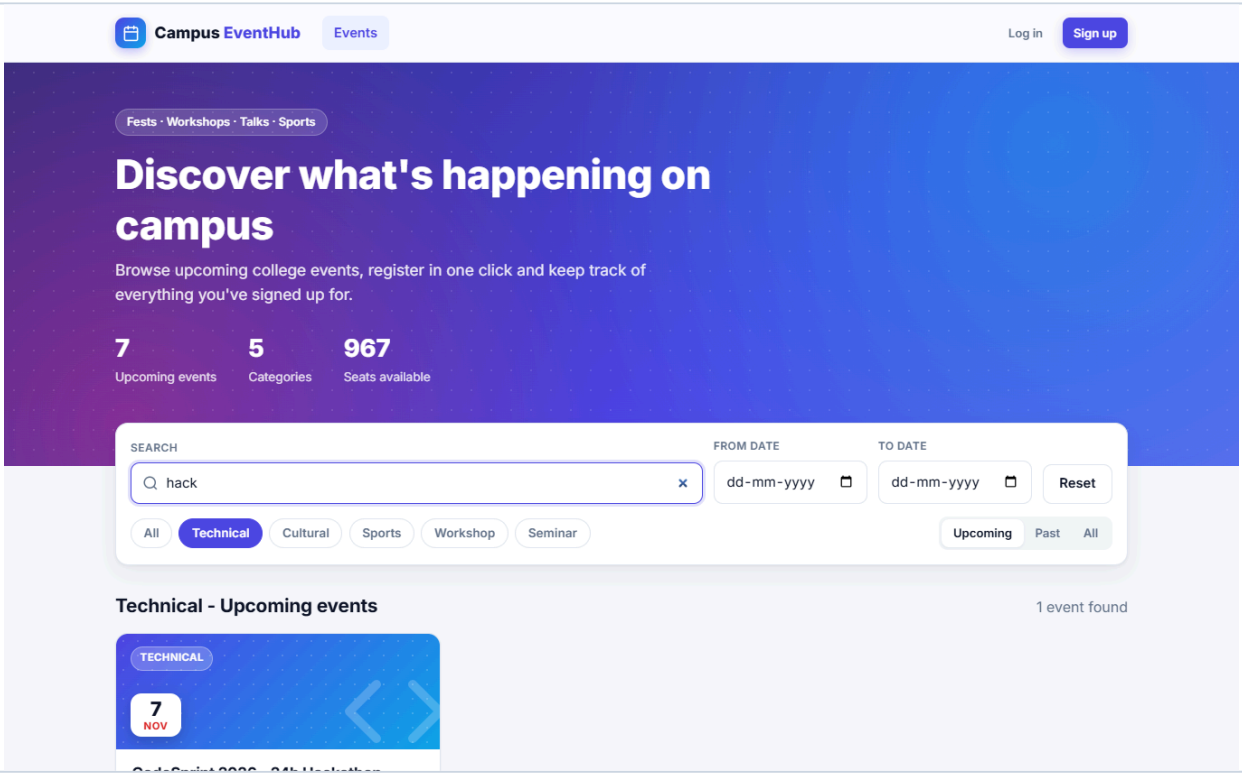


Figure 2: Searching "hack" within the Technical category

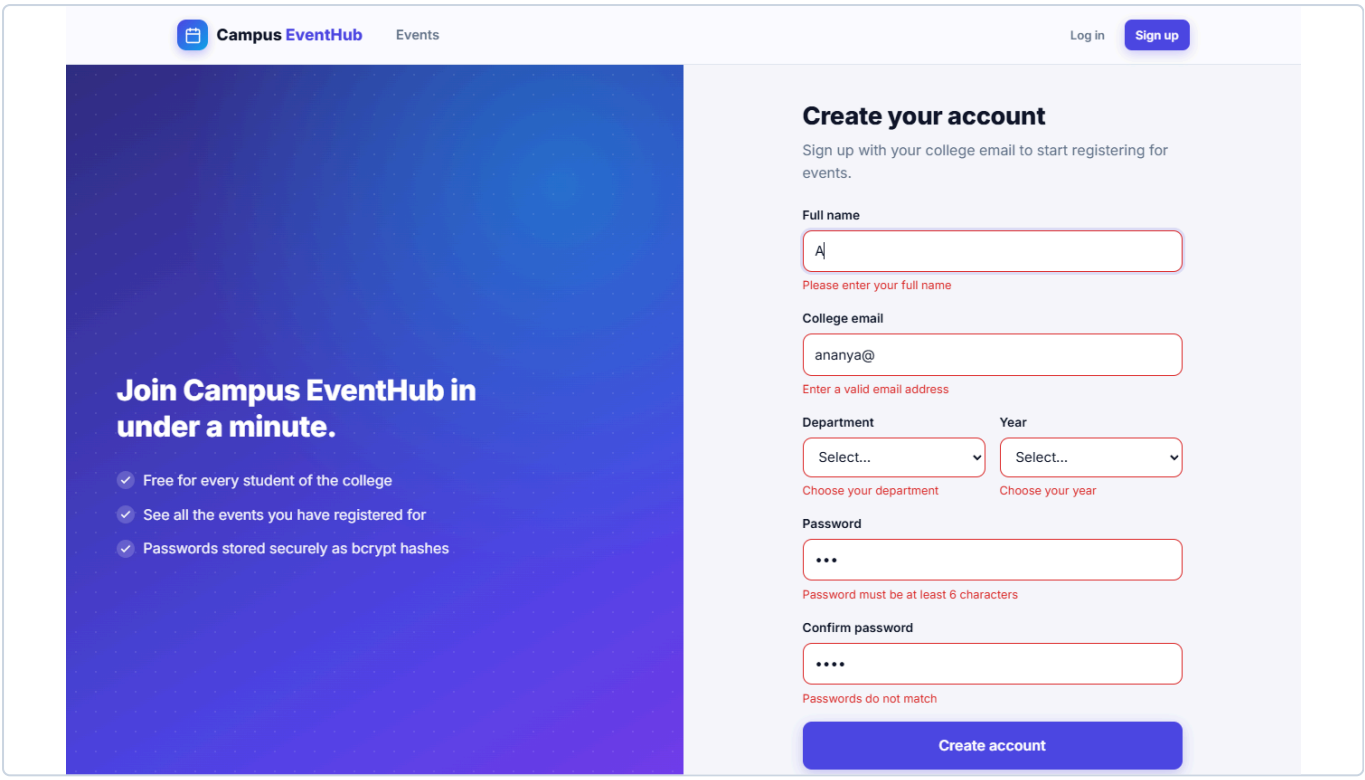


Figure 3: Sign-up form showing client-side validation errors

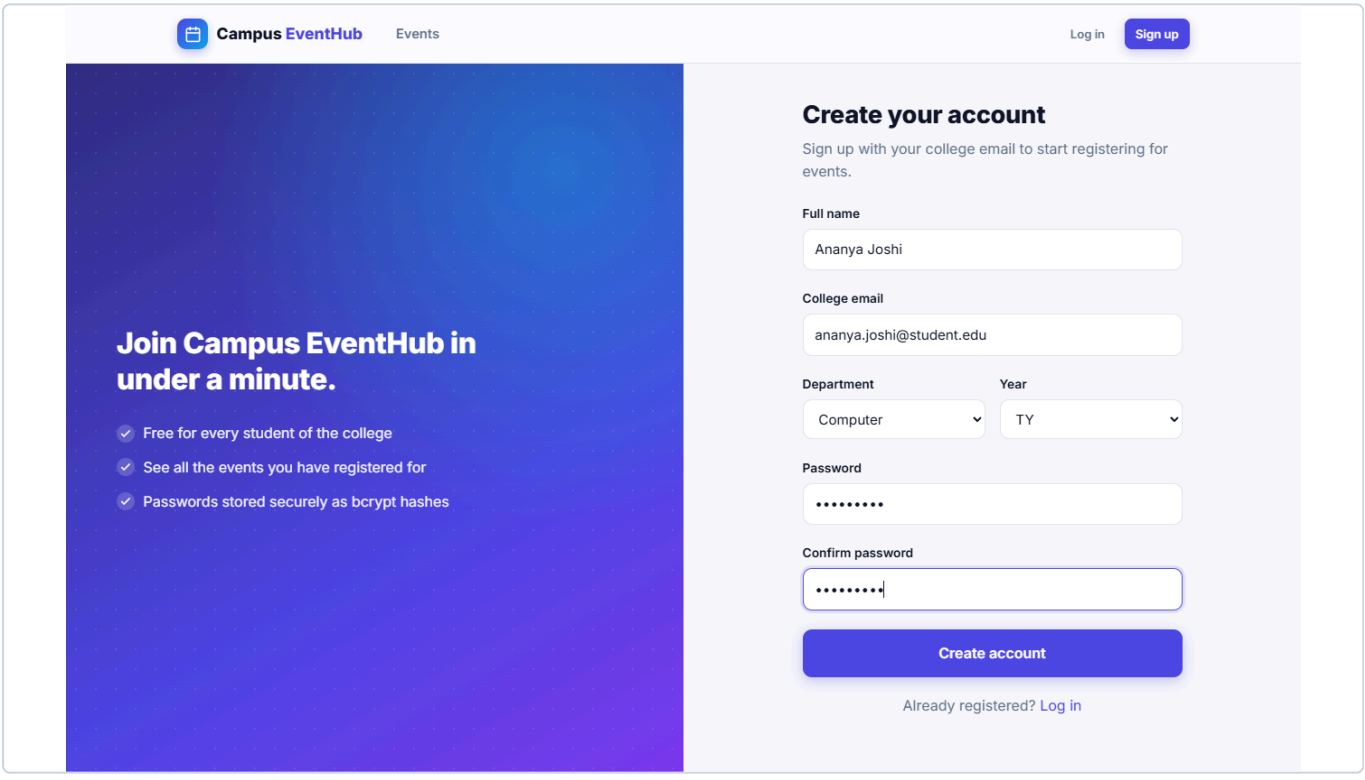


Figure 4: Sign-up form filled in by a new student

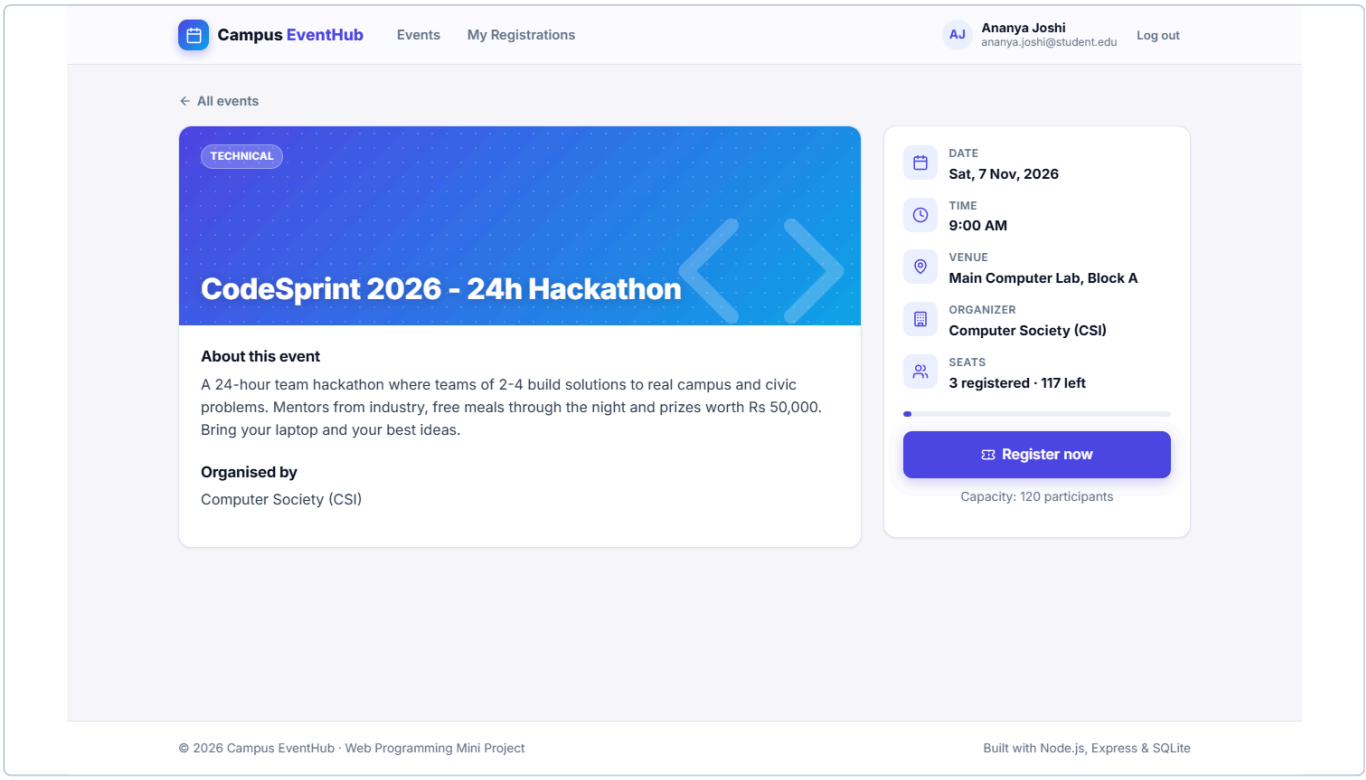


Figure 5: Event details page (logged in, before registering)

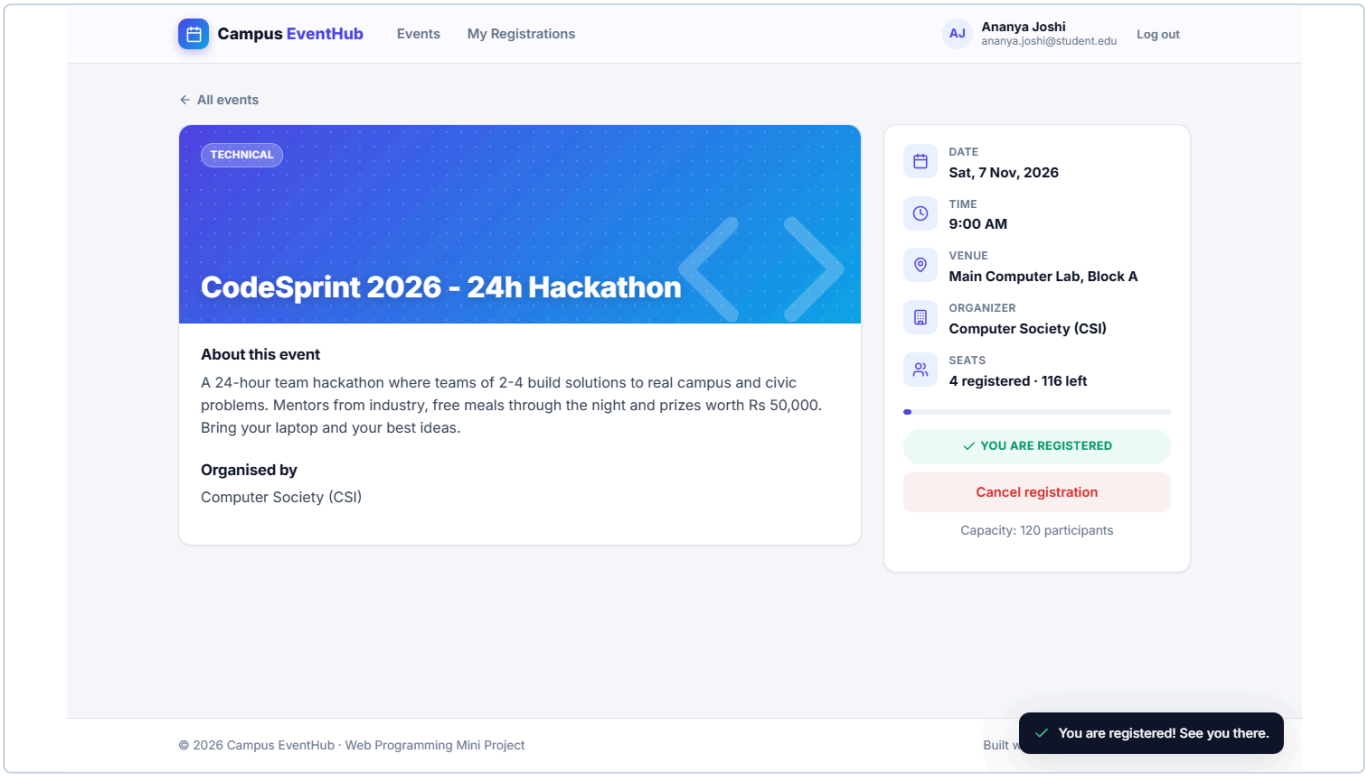


Figure 6: After clicking "Register now" - confirmation and updated seats

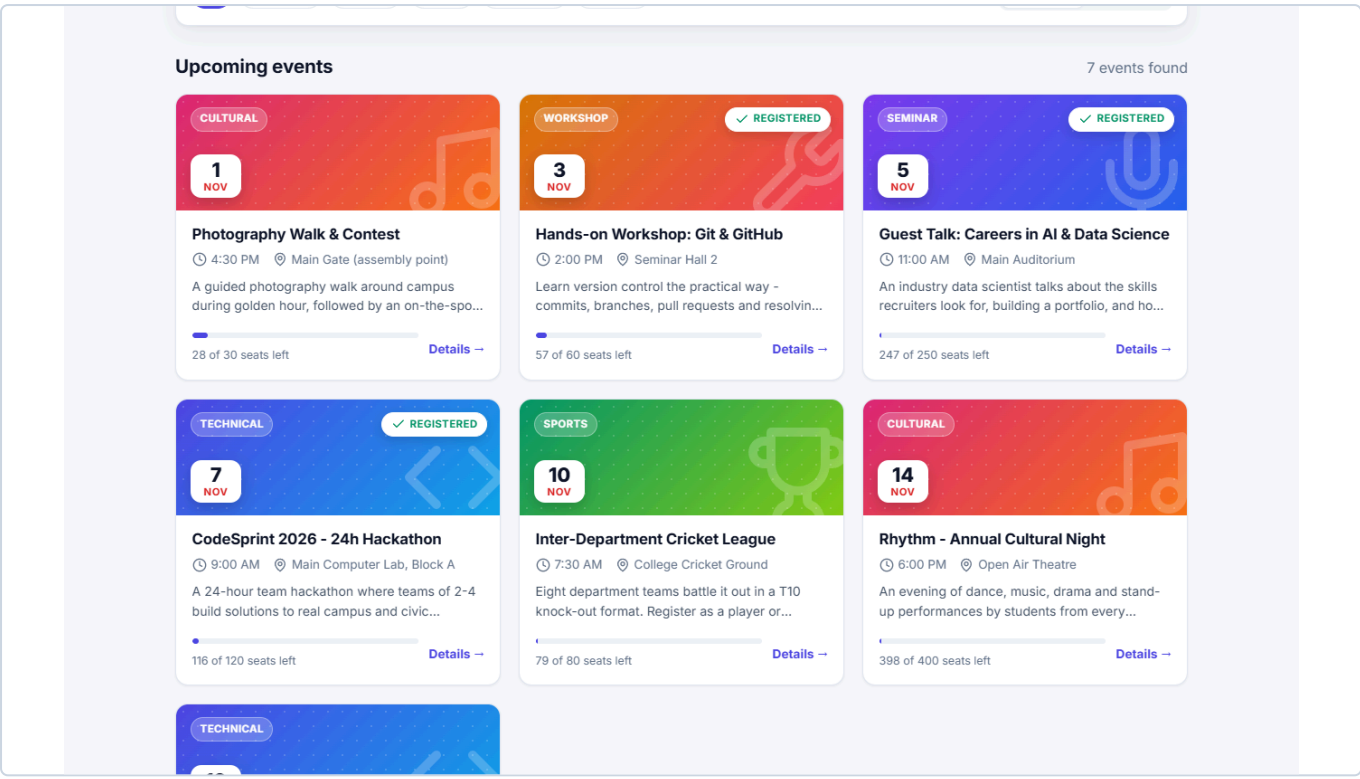


Figure 7: Home page when logged in - "Registered" badges on cards

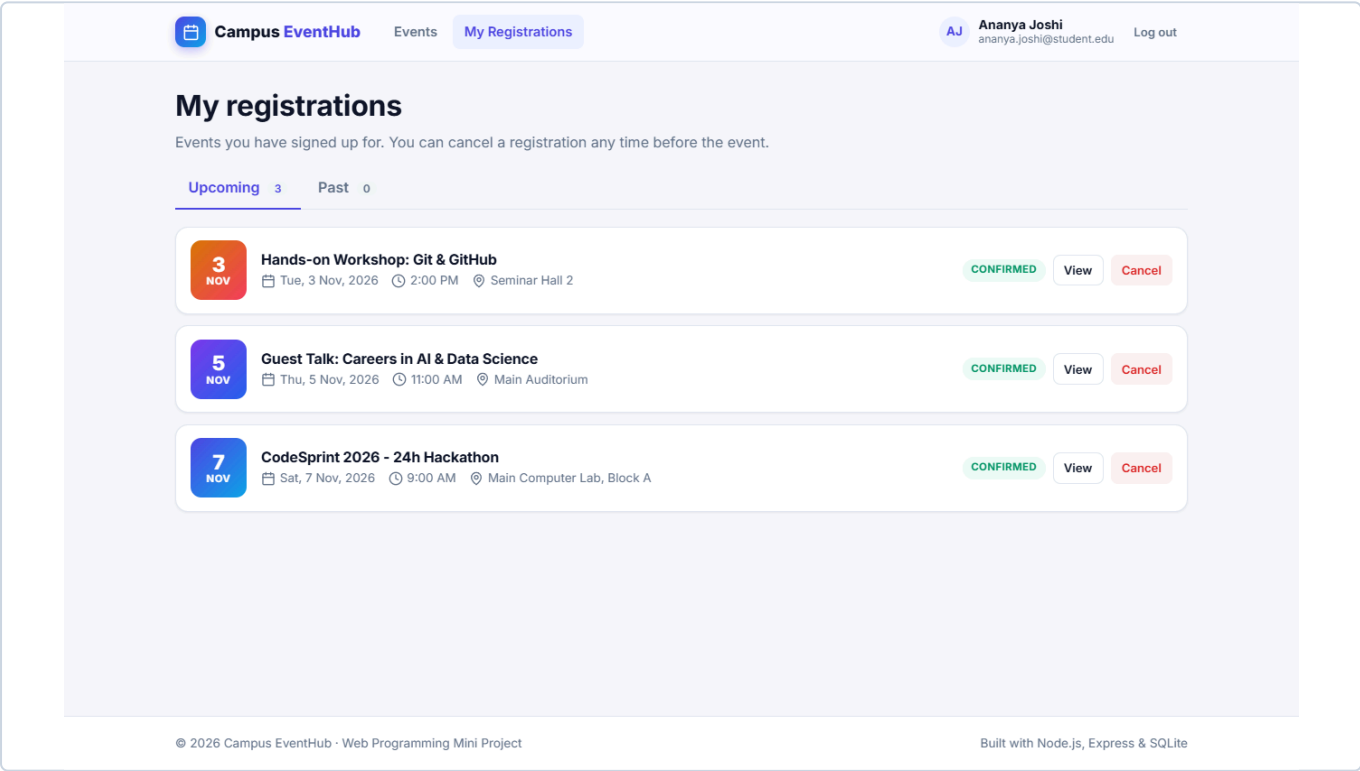


Figure 8: My Registrations page

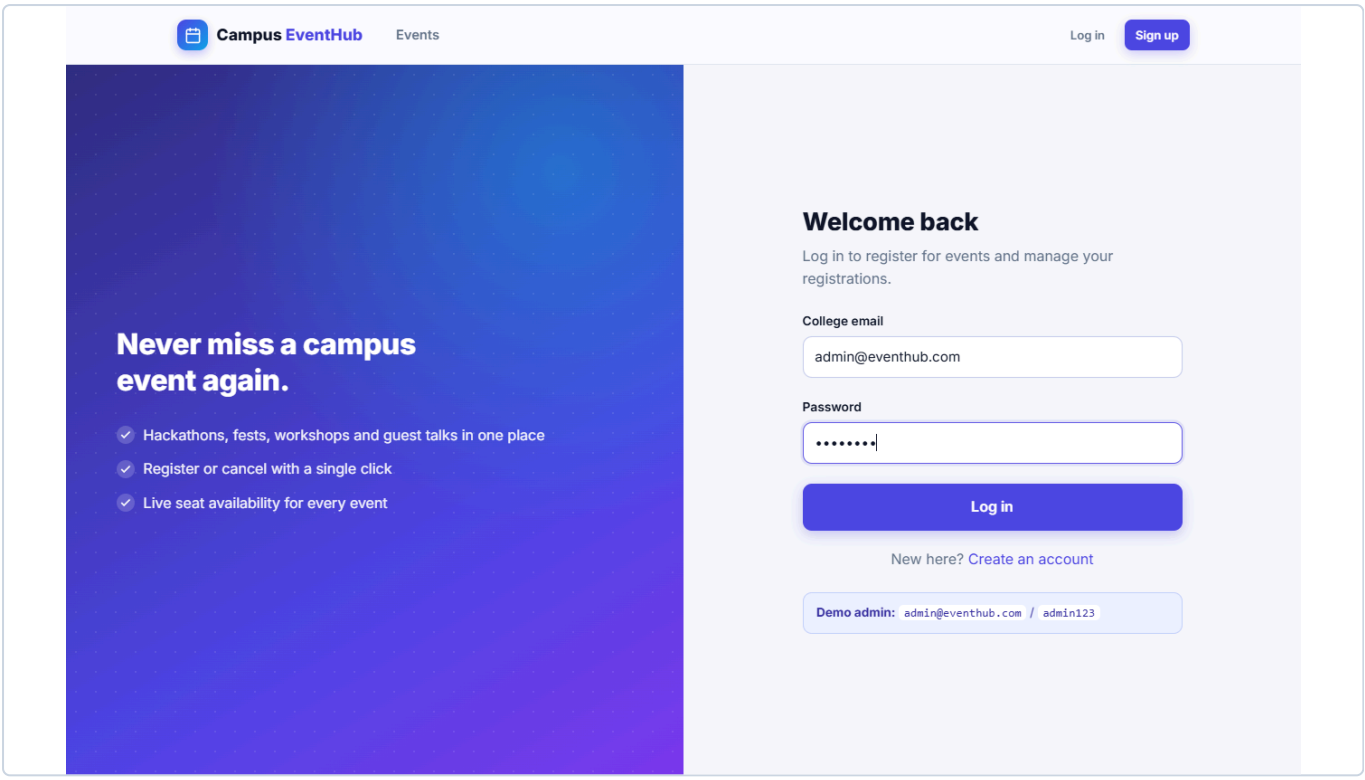


Figure 9: Login page with the admin credentials

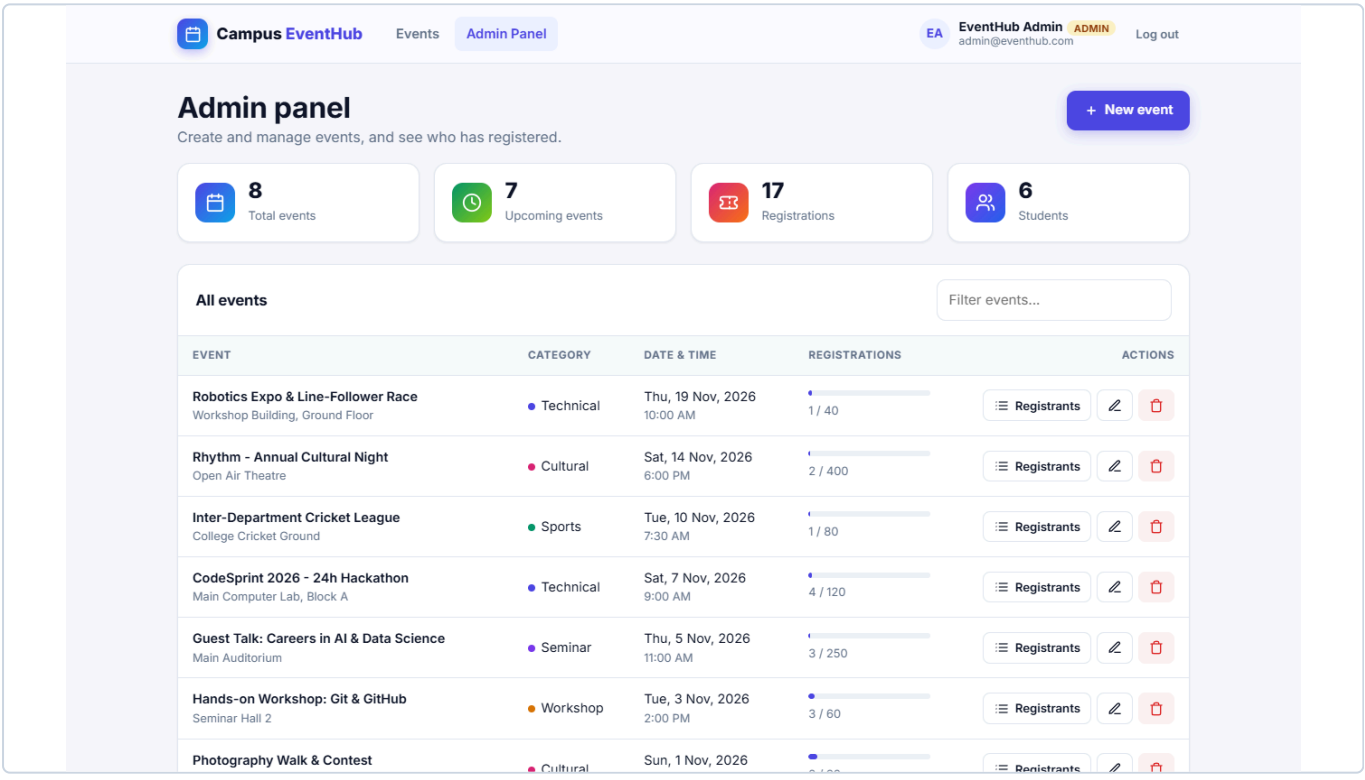


Figure 10: Admin panel - statistics and events table

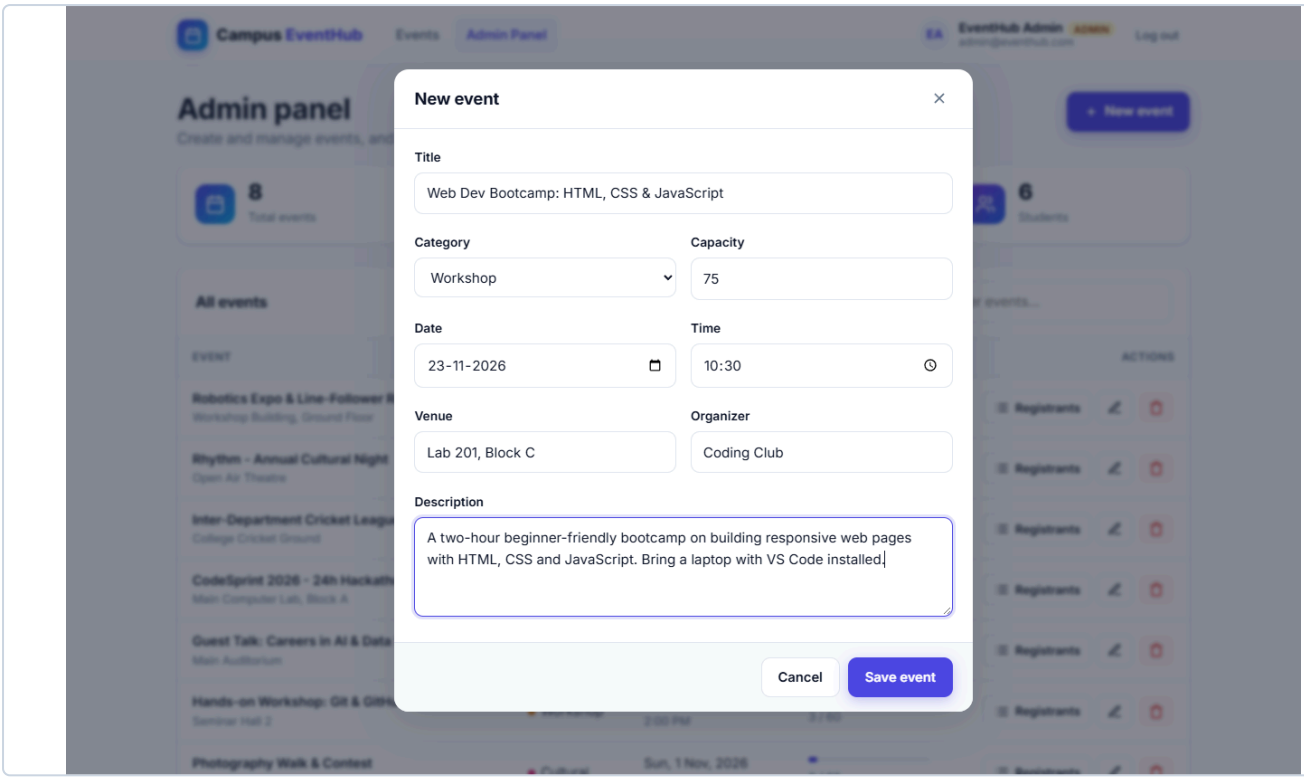


Figure 11: Admin creating a new event

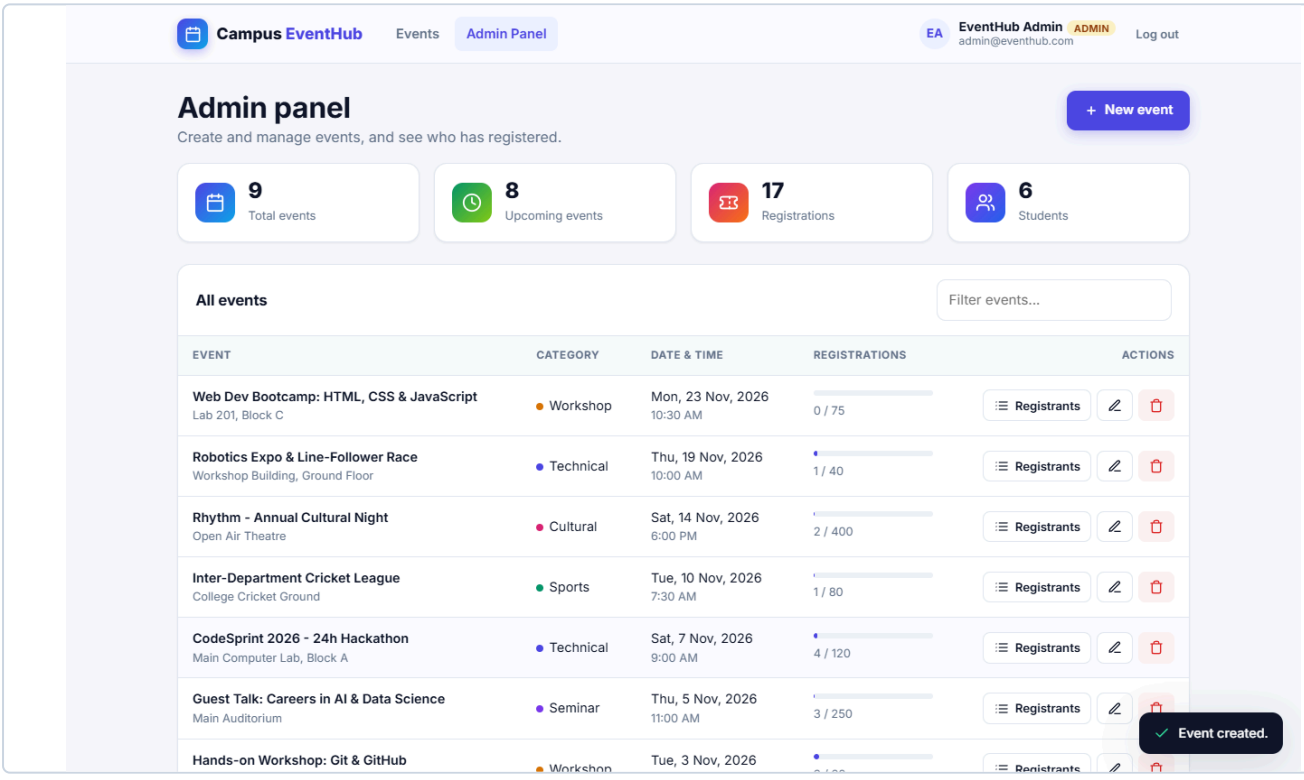


Figure 12: New event added (top row) with success message

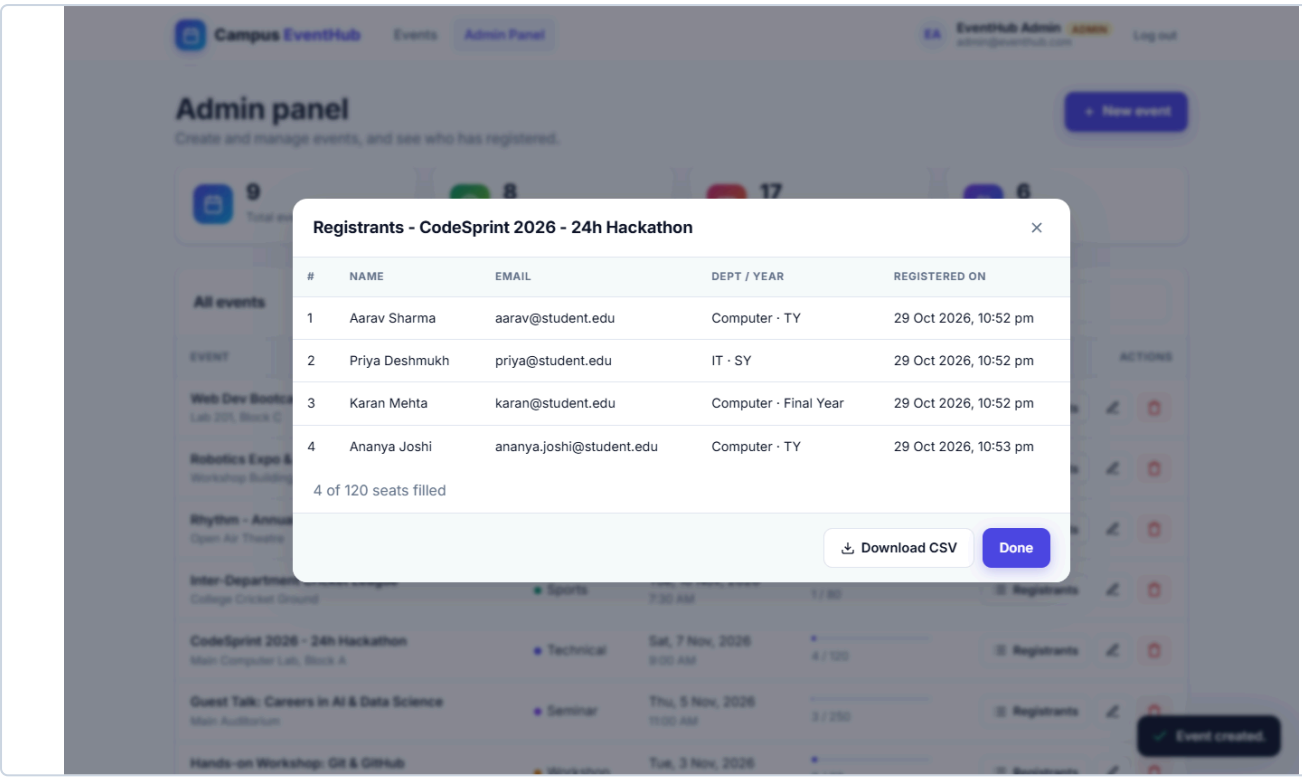


Figure 13: Registrant list for an event, with CSV download

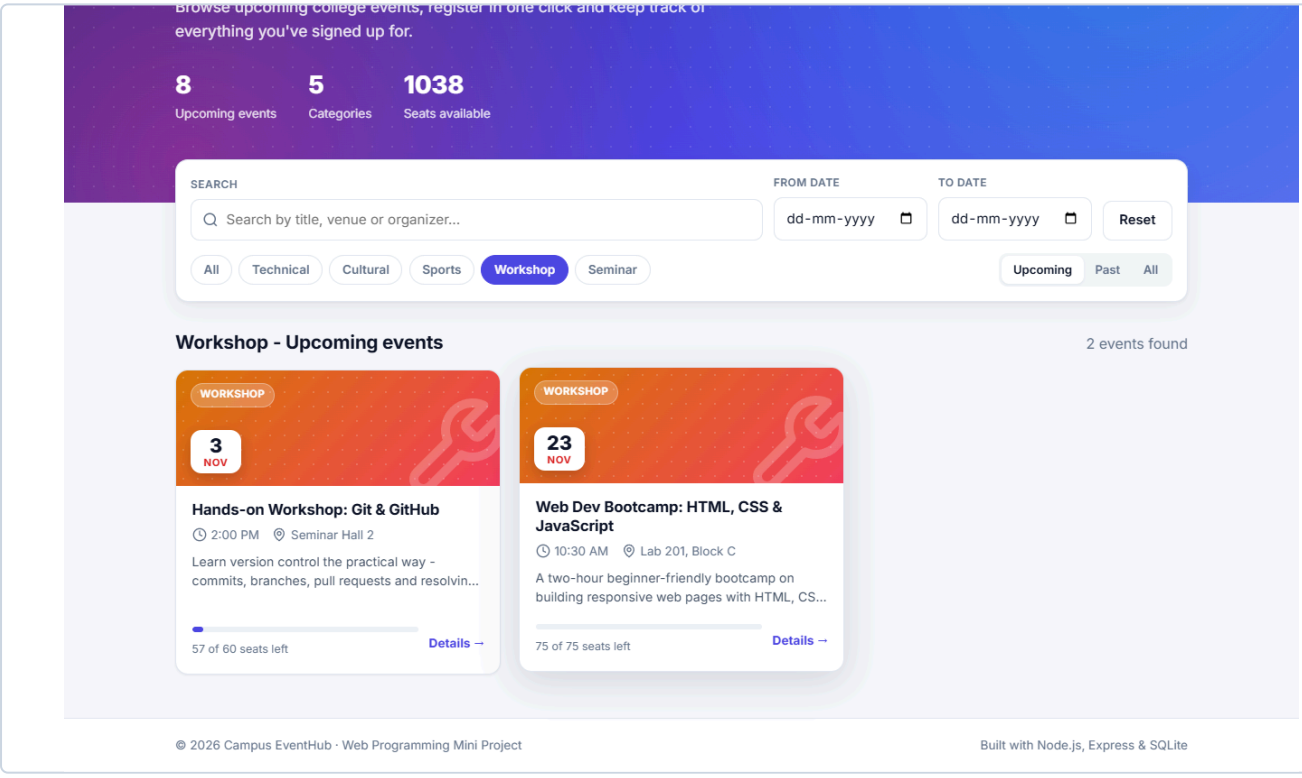


Figure 14: The new event is immediately visible to students

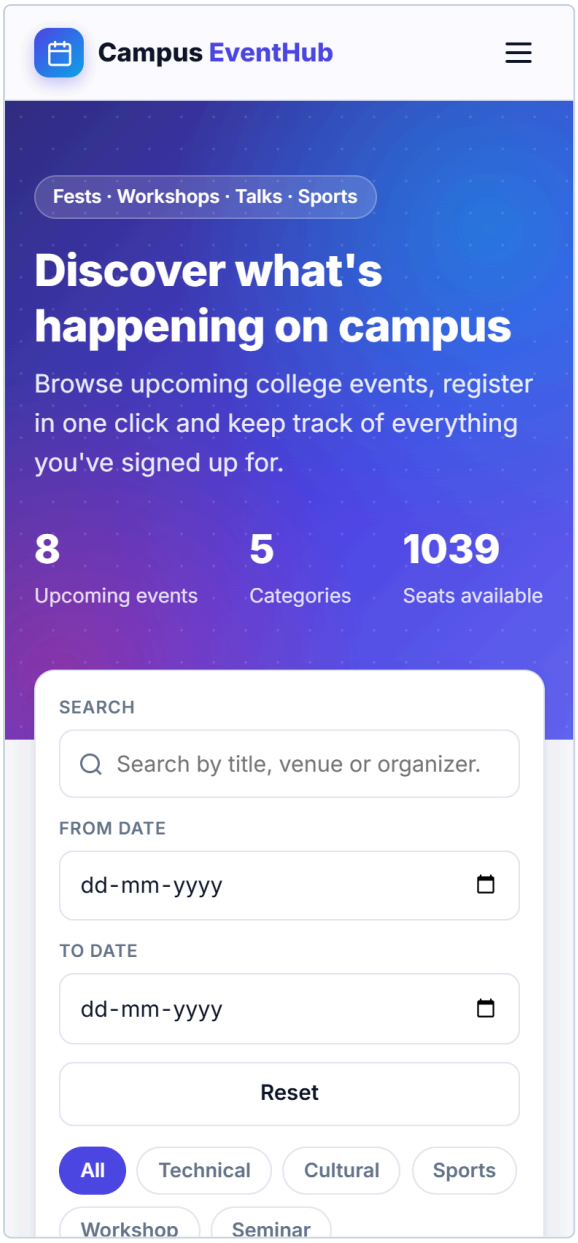


Figure 15: Responsive layout on a 390 px wide mobile screen

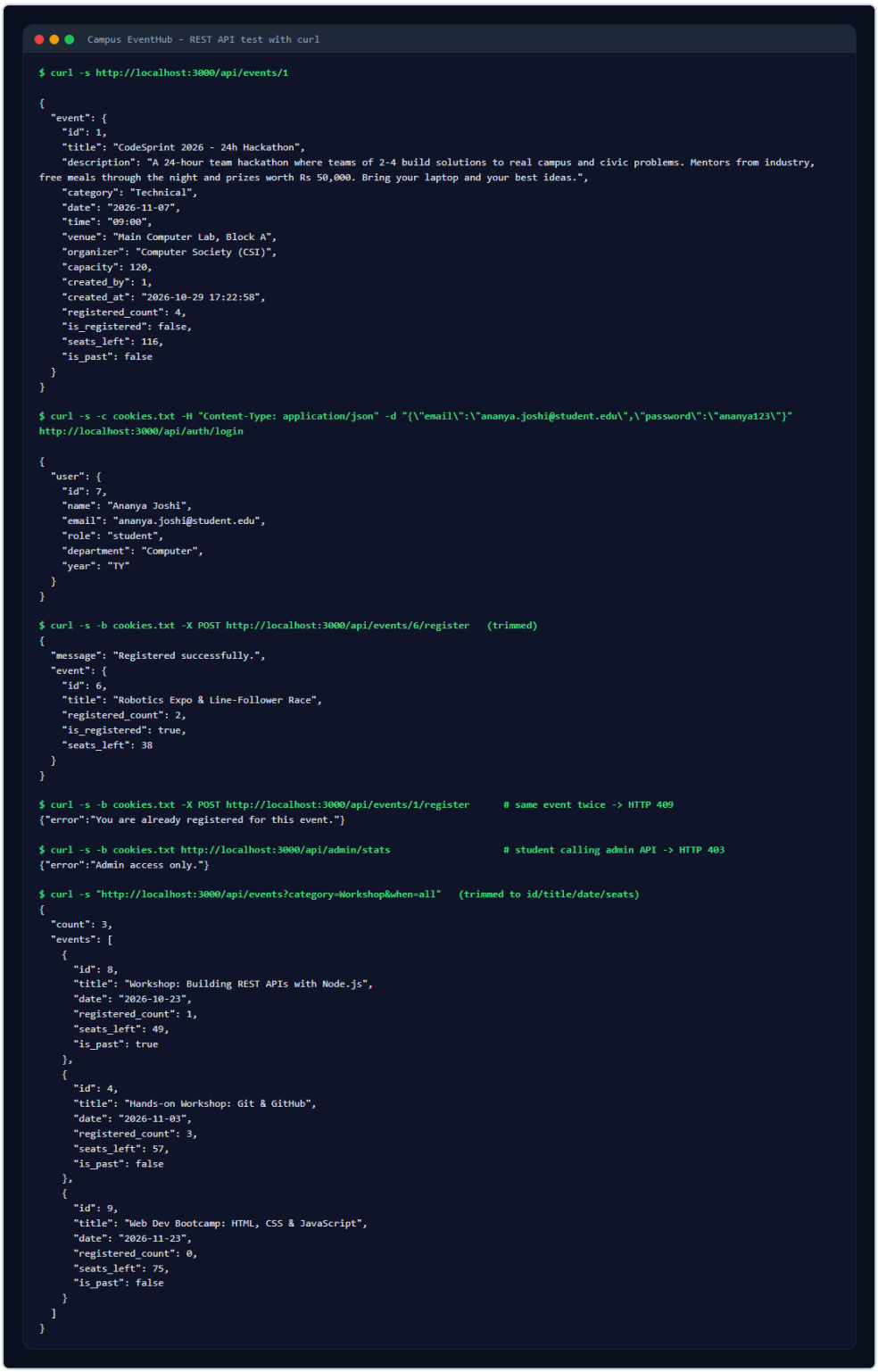


Figure 16: REST API tested from the command line with curl

11. Conclusion

Campus EventHub meets the goal of the mini project: it is a complete full-stack web application in which the browser (HTML, CSS, JavaScript) communicates with a Node.js / Express REST API that stores data in an SQLite database. Students can find and register for college events in a few clicks, and the admin can manage events and see registrations in real time.

Building it covered the core topics of web programming: semantic HTML forms, responsive CSS layouts with Flexbox and Grid, DOM manipulation and `fetch()` in JavaScript, HTTP methods and status codes, REST API design, server-side validation, relational database design with keys and constraints, and secure authentication with hashed passwords and cookies.

Future enhancements

- Email confirmation and reminders for registrations
- Event posters (image upload) and a waiting list when an event is full
- QR-code tickets for marking attendance at the venue
- Deployment to a cloud host with HTTPS and a custom domain